RESEARCH CONTRIBUTIONS

Applications: Image Processing and Computer Vision

Robert Haralick Editor

An Analysis and Algorithm for Polygon Clipping

YOU-DONG LIANG and BRIAN A. BARSKY University of California, Berkeley

ABSTRACT: The process of polygon clipping is analyzed and the understanding gained thereby forms the foundation of a very direct algorithm. Each edge of the input polygon is considered to be a vector and is then extended to be a line of infinite extent. The boundary line segments of the clip window are extended in a similar fashion to become boundary lines. The intersection points of an extended edge with the boundary lines are identified as entry or exit points. It is observed that these entry *and* exit *points can be used to characterize the visibility status of the edge. Not only might the edge have a visible segment, but it can also induce the inclusion in the output polygon of a vertex at a corner of the clip window called a* turning vertex. *To quantify the visibility conditions, the edge is written in parametric form. The possible contributions (including nil) that can be provided to the output polygon by a particular edge are established. This analysis is then embedded in an algorithm. An arbitrary example input polygon is given and the output of the algorithm is shown. The execution time for this example using this algorithm is measured to be half of what is required by the standard Sutherland–Hodgman reentrant polygon clipping algorithm.*

You-Dong Liang's primary research interest is computer-aided geometric design.

Brian Barsky's research interest include computer-aided geometric design and modeling, and interactive three-dimensional computer graphics.

Authors' Present Addresses: You-Dong Liang, Department of Mathematics, Zhejiang University, Hangzhou, Zhejiang, People's Republic of China; Brian A. Barsky, Berkeley Computer Graphics Laboratory, Computer Science Division, University of California, Berkeley, California 94720.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission. © 1983 ACM 0001-0782/83/1000-0868 75¢

1. INTRODUCTION

With the increasing popularity of raster graphics, it has become necessary to develop clipping algorithms that are capable of handling polygons. In Refs. [1] and [2], the authors describe a family of clipping algorithms for *lines*. Similar ideas are extended in the present paper to perform *polygon* clipping.

Polygon clipping is the process of removing those parts of a polygon that lie outside a region called the clip *window*. Both the input and output to a polygon clipping algorithm are polygons. Such an algorithm clips the input polygon to the boundaries of the clip window, thereby producing a clipped output polygon (Figure 1).

A major distinction between line and polygon clipping is that the output of the latter may involve segments of the window boundary in addition to parts of the input polygon. A particularly difficult case occurs when the input polygon surrounds a corner of the clip window; here, the output clipped polygon should contain two segments that meet at the corner (Figure 2).

These complications are naturally handled by the algorithm presented in this paper. As a result of a careful analysis of the process of polygon clipping, a very direct algorithm is developed.

2. KEY CONCEPTS

The algorithm assumes that the clip window is rectangular and coplanar with the input polygon. Both the input and output polygons are represented as an ordered sequence of vertices. The algorithm then operates on a list of input vertices $V_1V_2 \cdots V_n(V_{n+1} = V_1)$ and produces a list of output vertices $V'_1V'_2 \cdots V'_m(V'_{m+1} = V'_1)$, where m usually differs from n (and could even be zero if the entire polygon is outside the window).

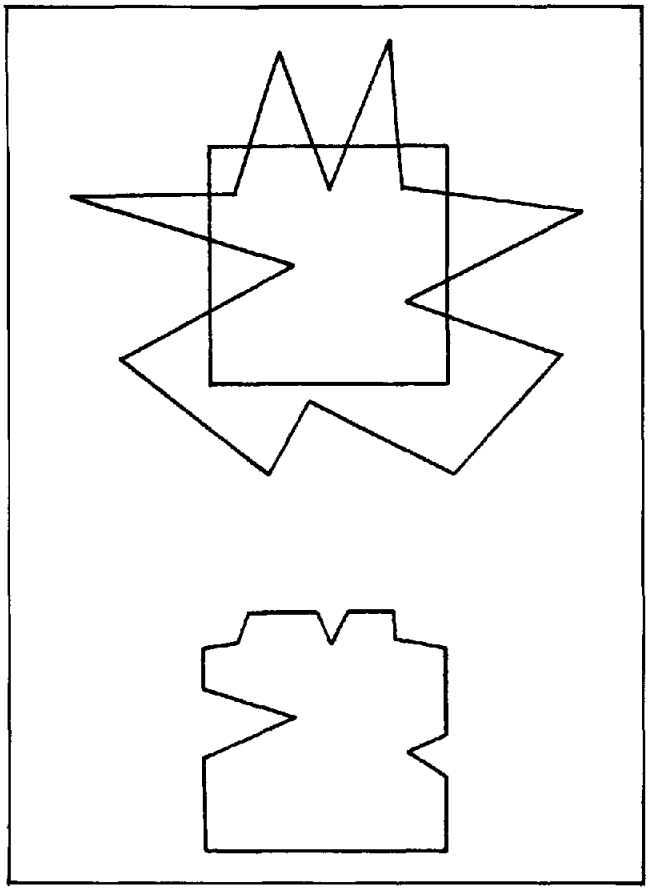

FIGURE 1. Input Polygon and Clipped Output Polygon.

The method considers each edge of the polygon independently. The ordering of the vertices provides a direction for each edge which can therefore be considered as a vector. To partition the polygon into a set of disjoint vectors, each vector is considered to be a half-open interval containing the head, but not the tail. The ith edge is $\overrightarrow{V_iV_{i+1}}$ which is a vector from V_i to V_{i+1}, as shown in Figure 3.

The vector $\overrightarrow{V_iV_{i+1}}$ is extended to be the directed line l_i of infinite extent having the same direction as $\overrightarrow{V_iV_{i+1}}$. Similarly, each of the four boundary line segments of the clip window is extended to be a line of infinite extent, called a *boundary line*. Each boundary line divides the plane into two regions;

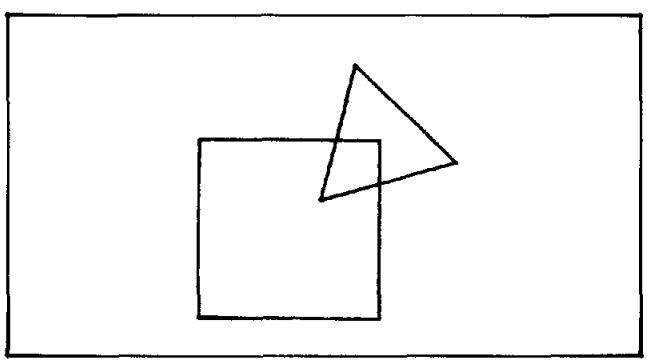

FIGURE 2. Input Polygon Surrounding a Corner of the Clip Window.

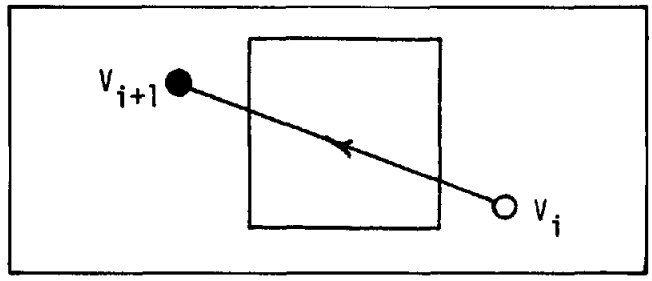

FIGURE 3. The ith Edge is the Vector from V_i to V_{i+1}.

we define the *visible side* to be that side on which the window lies, as shown in Figure 4.

The four boundary lines together divide the plane into nine regions. Each of these nine regions is on the visible side of some number, say ρ, of the boundary lines. The number ρ can be two or three or four. The nine regions and their associated ρ values are shown in Figure 5. The highest value of ρ is four and occurs in only one region; this region is the clip window. The lowest value of ρ is two and occurs in the four corner regions.

If the line l_i is not horizontal or vertical, it will intersect once with each of the boundary lines. (The case of a horizontal or vertical edge is discussed in Sec. 5.) A key observation is that at two of these intersection points the line l_i enters the visible region of the corresponding boundary line, while at the other two intersection points, it exits. These first two points are called *entry points* and the second two points are *exit points* (Figure 6). If l_i enters at a corner vertex of the window, the entry points are considered to be coincident, and similarly for l_i exiting at a corner vertex. Considering the ρ value of the regions through which l_i passes, this value increases by one at an entry point, and decreases by one at an exit point.

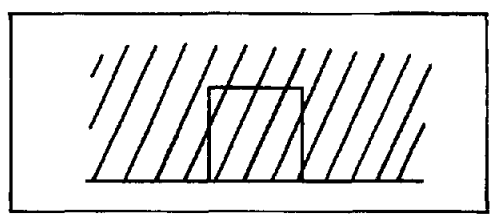

FIGURE 4. The Visible Side of a Boundary Line.

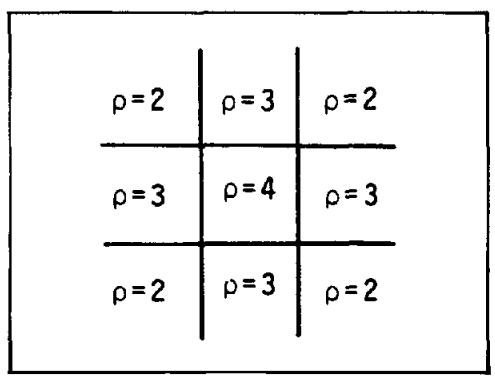

FIGURE 5. The Nine Regions of the Plane and their Associated ρ Values.

Further observations can be made about the entry and exit points. Assuming the line l_i is not horizontal or vertical, it must start in one of the corner regions and end in the diagonally opposite corner region. Therefore, it starts and ends with a ρ value of two. Moving along the direction of l_i, the ρ value will increase as l_i leaves the first corner region, and hence the first intersection point must be an entry point. Similarly, the ρ value will decrease as l_i arrives at the opposite corner region, and thus the last intersection point is always an exit point.

Recall that the clip window is the only region with a ρ value of four. Since the first intersection point is an entry point, in order for the line l_i to pass through the clip window the second intersection point must also be an entry point. Therefore, the identification of entry/exit attributes of the two middle points is equivalent to the determination of whether the line l_i passes through or bypasses the window. The former case occurs when the middle points are an entry point followed by an exit point [Figure 6(a)] while the latter case corresponds to an exit point followed by an entry point [Figure 6(b)].

The first case [Figure 6(a)] may or may not have a visible segment depending on where $\overrightarrow{V_iV_{i+1}}$ lies on l_i. In particular, to have a visible segment, $\overrightarrow{V_iV_{i+1}}$ cannot be entirely on one of the invisible sides of the window. This condition will be quantified for use in the algorithm. The second case [Figure 6(b)] will never have a visible segment. Nonetheless, l_i cannot be indiscriminately rejected as it would be for line clipping as it may still affect the output polygon depending on where $\overrightarrow{V_iV_{i+1}}$ lies on l_i.

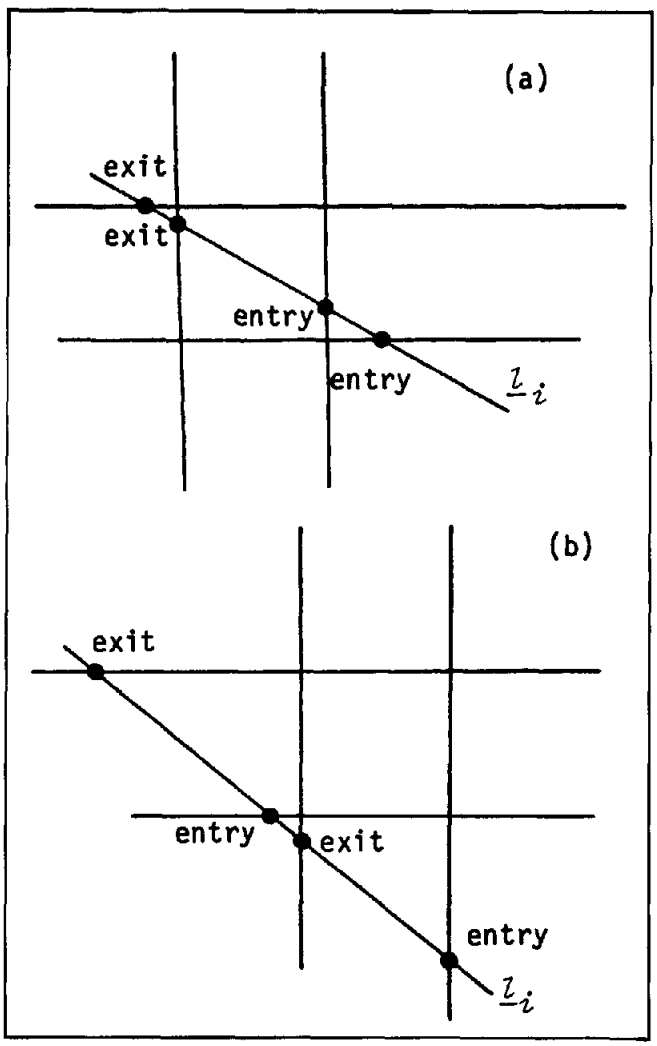

FIGURE 6. Entry and Exit Points: (a) l_i Passes Through Window; (b) l_i Bypasses Window.

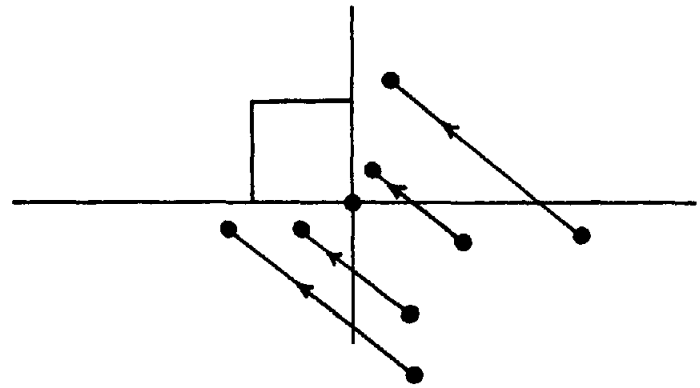

FIGURE 7. Turning Vertex due to First Entry Point.

In both cases, l_i can affect the output polygon even if it does not have a visible segment. This is what distinguishes polygon clipping from line clipping. A vertex at a corner of the clip window must sometimes be included in the output polygon. This occurs when the input polygon surrounds the corner. Such a corner vertex is called a *turning vertex* because the output polygon "turns" at this vertex. This situation can be detected by determining if an entry point of l_i occurs on $\overrightarrow{V_iV_{i+1}}$ and outside the clip window.

It should be noted that this is a necessary, but not sufficient condition for the polygon to surround the corner. For example, the entire polygon might lie outside the window. The accurate determination of this situation would require considering all the edges, not just one. Thus, this condition will sometimes generate extraneous turning vertices. For example, a turning vertex could be generated by an edge of a polygon even though the entire polygon lies outside the window.

When this condition holds, the corner vertex of the window that is closest to the relevant entry point is included in the output polygon (Figures 7 and 8). Note that a sufficient condition for the existence of a turning vertex is that $\overrightarrow{V_iV_{i+1}}$ contain the *first* entry point. This is because the first entry point cannot be on the clip window (unless it coincides with the second entry point at a corner vertex of the window) (Figure 7).

3. FORMALIZATION

As explained in the preceding section, several conclusions can be drawn from the identification of the entry/exit attribute of the intersection points along l_i with the boundary lines. Note that this attribute is independent of which boundary lines are horizontal or vertical. However, to formalize the notions presented in the previous section, it is convenient to introduce an x–y coordinate system for the window.

The coordinates of V_i and V_{i+1} are denoted by (x_i, y_i) and (x_{i+1}, y_{i+1}), respectively. The parametric equations for $\overrightarrow{V_iV_{i+1}}$ are

$$t = \frac{x - x_i}{\Delta x_i} = \frac{y - y_i}{\Delta y_i} \tag{1}$$

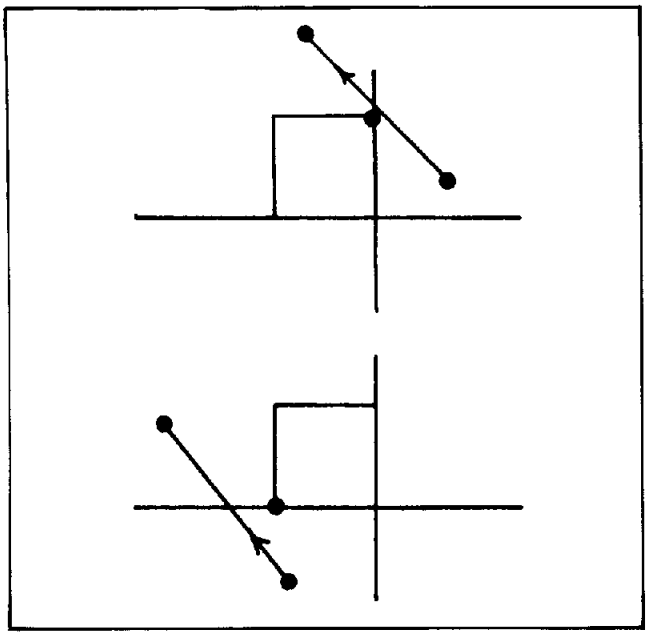

FIGURE 8. Turning Vertex due to Second Entry Point.

where $\Delta x_i = x_{i+1} - x_i$ and $\Delta y_i = y_{i+1} - y_i$. The parametric values corresponding to V_i and V_{i+1} are $t = 0$ and $t = 1$, respectively. If $0 < t \leq 1$, then the line segment $\overrightarrow{V_iV_{i+1}}$ is traced out; allowing $-\infty < t < \infty$ generates the line l_i.

If l_i points to the right, it will enter at the left vertical boundary and exit at the right vertical boundary, and conversely for l_i pointing to the left. Similarly, if l_i points up, it will enter at the bottom horizontal boundary and exit at the top horizontal boundary, and conversely for l_i pointing down.

To be more specific, consider the following inequalities that define the interior of the clip window (Fig. 9).

$$x_{left} \leq x \leq x_{right}$$
$$y_{bottom} \leq y \leq y_{top}$$

Now, the vertical boundary line containing an entry point will be denoted $x = x_{in}$, and $x = x_{out}$ will denote the vertical boundary line containing an exit point. The analogous horizontal boundary lines will be denoted $y = y_{in}$ and $y = y_{out}$, respectively. From the preceding discussion,

$$x_{in} = \begin{cases} x_{left} & \Delta x_i > 0 \\ x_{right} & \Delta x_i \leq 0 \end{cases}$$

$$x_{out} = \begin{cases} x_{right} & \Delta x_i > 0 \\ x_{left} & \Delta x_i \leq 0 \end{cases}$$

$$y_{in} = \begin{cases} y_{bottom} & \Delta y_i > 0 \\ y_{top} & \Delta y_i \leq 0 \end{cases}$$

$$y_{out} = \begin{cases} y_{top} & \Delta y_i > 0 \\ y_{bottom} & \Delta y_i \leq 0 \end{cases} \qquad (2)$$

From Eq. 1, the corresponding parametric values are as follows:

$$t_{in_x} = (x_{in} - x_i)/\Delta x_i, \quad \Delta x_i \neq 0$$

$$t_{out_x} = (x_{out} - x_i)/\Delta x_i, \quad \Delta x_i \neq 0$$

$$t_{in_y} = (y_{in} - y_i)/\Delta y_i, \quad \Delta y_i \neq 0$$

$$t_{out_y} = (y_{out} - y_i)/\Delta y_i, \quad \Delta y_i \neq 0 \qquad (3)$$

These parametric values can then be used to easily determine the parametric values of the first and second entry points and the first and second exit points, which will be denoted t_{in_1}, t_{in_2}, t_{out_1}, and t_{out_2}, respectively:

$$t_{in_1} = \min(t_{in_x}, t_{in_y})$$

$$t_{in_2} = \max(t_{in_x}, t_{in_y})$$

$$t_{out_1} = \min(t_{out_x}, t_{out_y})$$

$$t_{out_2} = \max(t_{out_x}, t_{out_y}) \qquad (4)$$

4. CLASSIFICATION

Using the notation introduced in Sec. 3, this section classifies what contributions, if any, a particular edge of the input polygon will make to the output polygon. For a given edge of the input polygon, there are three cases where it will have no contribution to the output polygon, and hence can be rejected. The first of these occurs when the edge terminates prior to its first entry point ($1 < t_{in_1}$). The region of the plane where this occurs is shown in Figure 10 for all orientations of l_i. The second case is characterized by an edge that begins after its first entry point ($0 \geq t_{in_1}$) and terminates prior to its

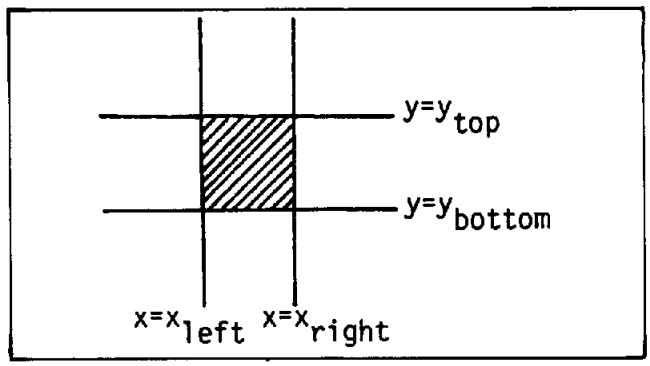

FIGURE 9. The Interior of the Clip Window.

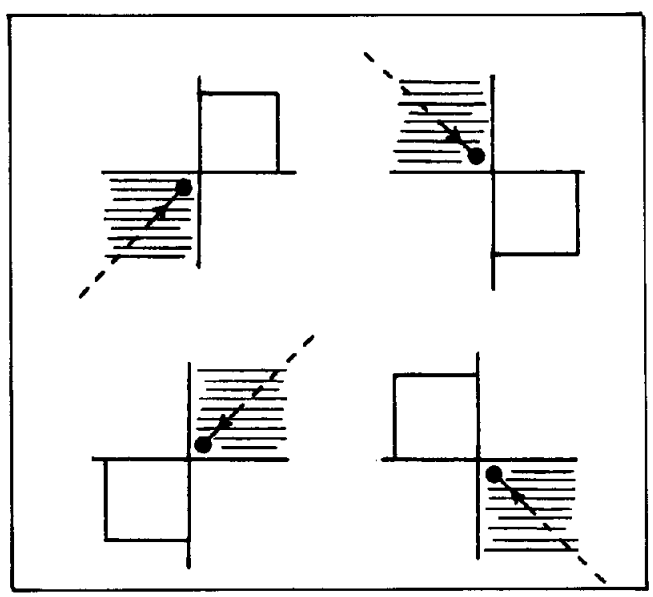

FIGURE 10. Case 1 ($1 < t_{in_1}$): Reject.

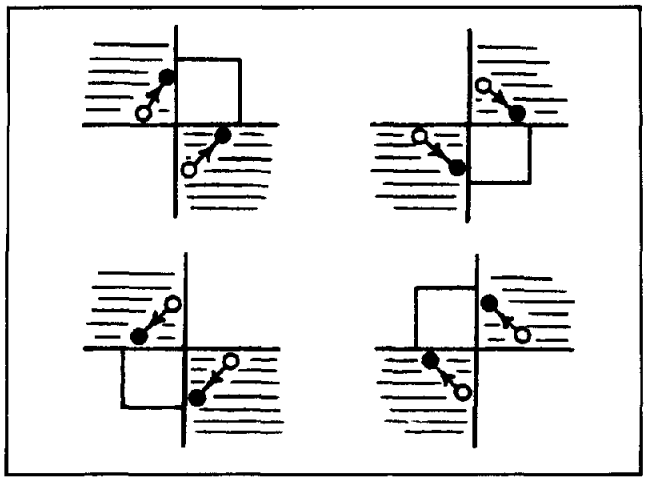

FIGURE 11. Case 2 ($0 \geq t_{in_1}$ and $1 < t_{in_2}$): Reject.

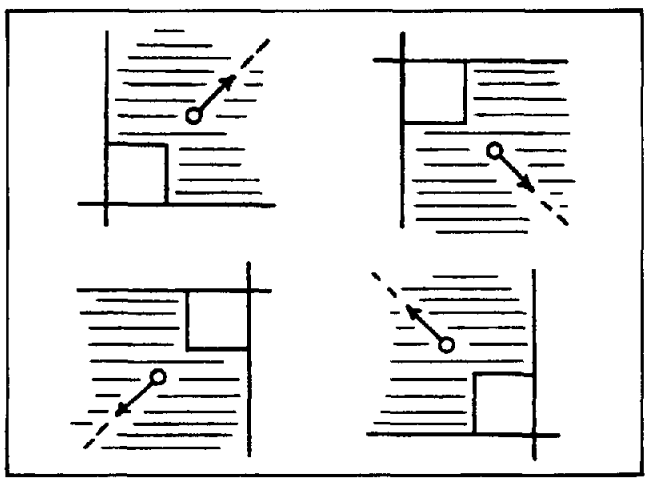

FIGURE 12. Case 3 ($0 \geq t_{in_2}$ and $0 \geq t_{out_1}$): Reject.

second one ($1 < t_{in_2}$), as shown in Figure 11. The third case applies to an edge if it begins after its second entry point ($0 \geq t_{in_2}$) and first exit point ($0 \geq t_{out_1}$), which is illustrated in Figure 12.

The above three cases completely account for all the rejection possibilities. Consider now the cases where there is a contribution to the output polygon. The first of these is the case where there is a visible segment, as would be found in line clipping. As indicated in Sec. 2, this will occur if and only if the second entry point precedes the first exit point on l_i ($t_{in_2} \leq t_{out_1}$), and $\overrightarrow{V_iV_{i+1}}$ begins prior to the first exit point ($0 < t_{out_1}$) and terminates after the second entry point ($1 \geq t_{in_2}$) (Figure 13). Note that it is still possible for each of the endpoints of $\overrightarrow{V_iV_{i+1}}$ to be outside or inside the window. If an endpoint is outside the window, then the corresponding endpoint of the visible segment is the intersection point with the window boundary.

Consider the case where V_i is outside the window. Since $\overrightarrow{V_iV_{i+1}}$ begins prior to the first exit point, the only way that V_i will be outside the window is that $\overrightarrow{V_iV_{i+1}}$ begin prior to the second entry point ($0 < t_{in_2}$) (Figure 14). This intersection point will be on the same boundary as the second entry point. Thus, the intersection point will be on a vertical boundary if $t_{in_x} > t_{in_y}$ and on a horizontal boundary if $t_{in_y} > t_{in_x}$.

To derive the appropriate expression for the intersection point, the parametric equations (1) for $\overrightarrow{V_iV_{i+1}}$ are rewritten in the following form.

$$x = x_i + t\Delta x_i$$
$$y = y_i + t\Delta y_i \qquad (5)$$

Then the intersection point on a vertical boundary is

$$x = x_i + t_{in_x}\Delta x_i$$
$$y = y_i + t_{in_x}\Delta y_i \qquad (6)$$

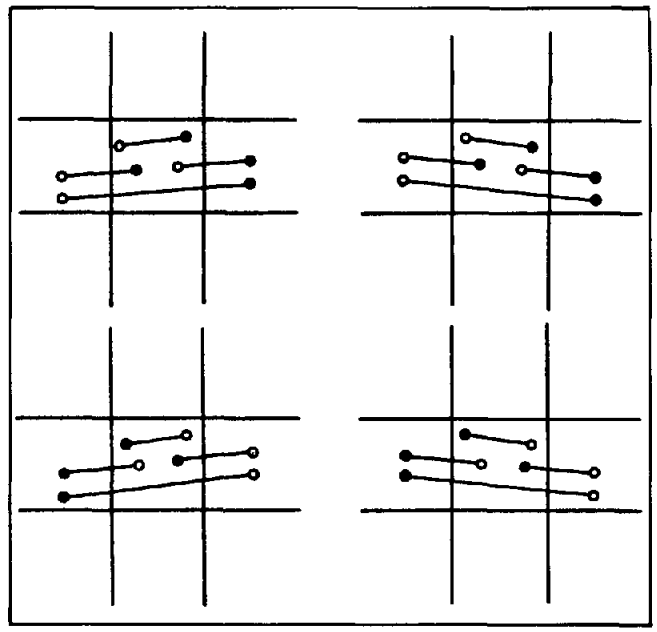

FIGURE 13. Case 4 ($t_{in_2} \leq t_{out_1}$ and $0 < t_{out_1}$ and $1 \geq t_{in_2}$): Visible Segment.

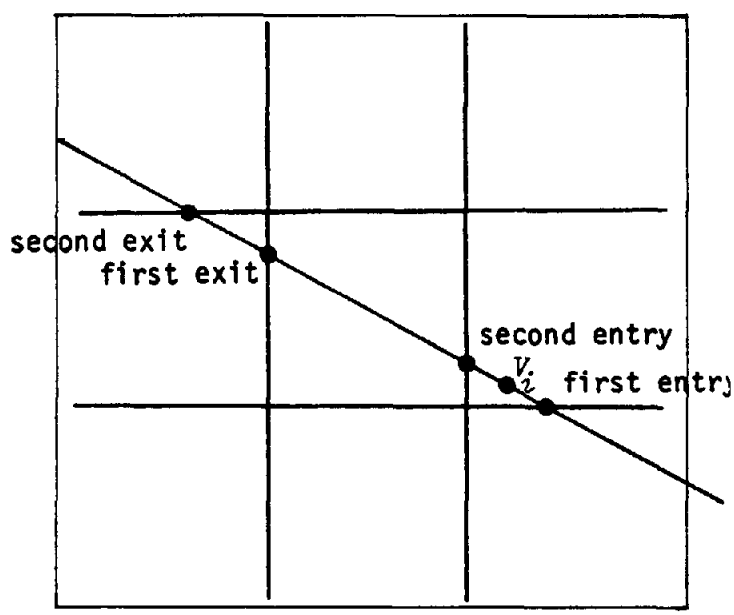

FIGURE 14. $\overrightarrow{V_iV_{i+1}}$ Begins Prior to the Second Entry Point.

and on a horizontal boundary it is

$$x = x_i + t_{in_y}\Delta x_i$$

$$y = y_i + t_{in_y}\Delta y_i \tag{7}$$

These equations can be simplified by recalling Eq. (3): the expression for a vertical boundary becomes

$$x = x_{in}$$

$$y = y_i + t_{in_x}\Delta y_i \tag{8}$$

and for a horizontal boundary it becomes

$$x = x_i + t_{in_y}\Delta x_i$$

$$y = y_{in} \tag{9}$$

The case where V_{i+1} is outside the window can be analyzed in an analogous fashion. $\overrightarrow{V_iV_{i+1}}$ must terminate after the first exit point ($1 > t_{out_1}$). The intersection point will be on a vertical boundary if $t_{out_x} < t_{out_y}$ and on a horizontal boundary if $t_{out_y} < t_{out_x}$. The expression for the intersection point on a vertical boundary is then

$$x = x_{out}$$

$$y = y_i + t_{out_x}\Delta y_i \tag{10}$$

and on a horizontal boundary it is

$$x = x_i + t_{out_y}\Delta x_i$$

$$y = y_{out} \tag{11}$$

If an endpoint is inside the window, it will also be an endpoint of the visible segment. However, there is no need to add the first endpoint (V_i) to the output polygon because it will have already been added from the previous edge. (In the case of the first edge, this endpoint will be appended from the closing edge.) The second endpoint (V_{i+1}), though, must be added to the output polygon when this endpoint is inside the window. Since V_iV_{i+1} terminates after the second entry point, a sufficient condition for this to occur is that V_iV_{i+1} terminate prior to the first exit point ($1 < t_{out_1}$). In this case, the contribution to the output polygon is just V_{i+1}.

The final two cases handle contributions to the output polygon other than a visible segment. These contributions take the form of turning vertices. As explained in Sec. 2, these situations occur when an entry point of I_i lies on $\overrightarrow{V_iV_{i+1}}$ and is outside the clip window. The two cases correspond to this entry point being either the first or second entry point, respectively.

Consider first the case where it is the first entry point. Since the first entry point cannot be on the clip window (except at a corner vertex), a sufficient condition for the occurrence of this case is that the first entry point be on $\overrightarrow{V_iV_{i+1}}$ ($0 < t_{in_1} \leq 1$). This is shown in Figure 15 for all orientations of I_i. The turning vertex is (x_{in}, y_{in}) since this is the corner vertex of the window that is closest to the first entry point. This turning vertex, then, is added to the output polygon.

The final case is that of a turning vertex due to the second entry point. This arises when the second entry point lies on $\overrightarrow{V_iV_{i+1}}$ ($0 < t_{in_2} \leq 1$) and is outside the clip window ($t_{out_1} < t_{in_2}$) (Figure 16). Unlike the previous case, there are now two possibilities for the turning vertex. This vertex may be one of two corner vertices, depending on whether the second entry point is on a vertical or horizontal boundary. In the vertical case ($t_{in_x} > t_{in_y}$), the closest corner vertex to the second entry point is (x_{in}, y_{out}), while the horizontal case ($t_{in_y} > t_{in_x}$) yields the turning vertex (x_{out}, y_{in}). From this, the

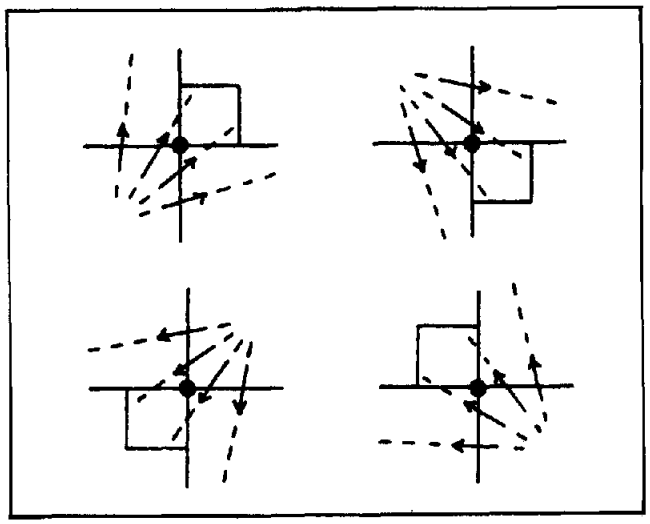

FIGURE 15. Case 5 ($0 < t_{in_1} \leq 1$): Turning Vertex.

turning vertex is determined and then added to the output polygon.

These six cases for clipping a polygon edge are summarized in Table I. These cases are mutually exclusive with the exception of Case 5 (turning vertex due to the first entry point). After creating this turning vertex, the vector $\overrightarrow{V_iV_{i+1}}$ might continue to generate a visible segment (Case 4) or a turning vertex due to the second entry point (Case 6). (Cases 4 and 6 cannot both occur, since they are mutually exclusive.) Note that the output of Case 5 must precede that of Case 4 or Case 6. This ordering is important and must be reflected in the algorithm. Thus, the contribution to the output polygon from an edge is composed of a turning vertex due to the first entry point (if any) and then either a visible segment (if any) or a turning vertex due to the second entry point (if any), but *not* both. The possibilities are summarized in Table II. (Recall that a visible segment is usually output as a sequence of two vertices, except that the first endpoint (V_i) is not added to the output polygon if it is inside the window.)

TABLE I. Classification of the Six Cases for Clipping a Polygon Edge

Case	Condition	Contribution	Figure
1	$1 < t_{in_1}$	none	10
2	$0 \geq t_{in_1}$ and $1 < t_{in_2}$	none	11
3	$0 \geq t_{in_2}$ and $0 \geq t_{out_1}$	none	12
4	$t_{in_2} \leq t_{out_1}$ and $0 < t_{out_1}$ and $1 \geq t_{in_2}$	visible segment	13
5	$0 < t_{in_1} \leq 1$	turning vertex	15
6	$0 < t_{in_2} \leq 1$ and $t_{out_1} < t_{in_2}$	turning vertex	16

TABLE II. Possible Contribution to the Output Polygon from an Edge

Cases	Contribution
5	Turning vertex due to first entry point
5 then 4	Turning vertex due to first entry point followed by visible segment
5 then 6	Turning vertex due to first entry point followed by turning vertex due to second entry point
4	Visible segment
6	Turning vertex due to second entry point

5. HORIZONTAL/VERTICAL EDGES

It has been assumed that l_i is neither horizontal nor vertical. Of course, these situations can occur, and for completeness they will now be addressed. It will be shown that results of the previous section can also be applied to horizontal and vertical edges. This is achieved by carefully assigning positive and negative values of infinity to the parametric values corresponding to the entry and exit points. As for the oblique orientation, both the cases of l_i passing through and bypassing the window will be considered.

First, the case of l_i passing through the window will be considered. A horizontal (or vertical) orientation can be treated as a limiting case of the oblique orientation. The near-horizontal case is shown in Figure 17 and can be characterized by the condition.

$$t_{in_y} \leq t_{in_x} \leq t_{out_x} \leq t_{out_y} \tag{12}$$

As l_i approaches the horizontal, t_{in_y} and t_{out_y} cease to be finite. Since t_{in_x} and t_{out_x} are finite, the infinite values of t_{in_y} and t_{out_y} must be as follows:

$$\begin{aligned} t_{in_y} &\rightarrow -\infty \\ t_{out_y} &\rightarrow +\infty \end{aligned} \tag{13}$$

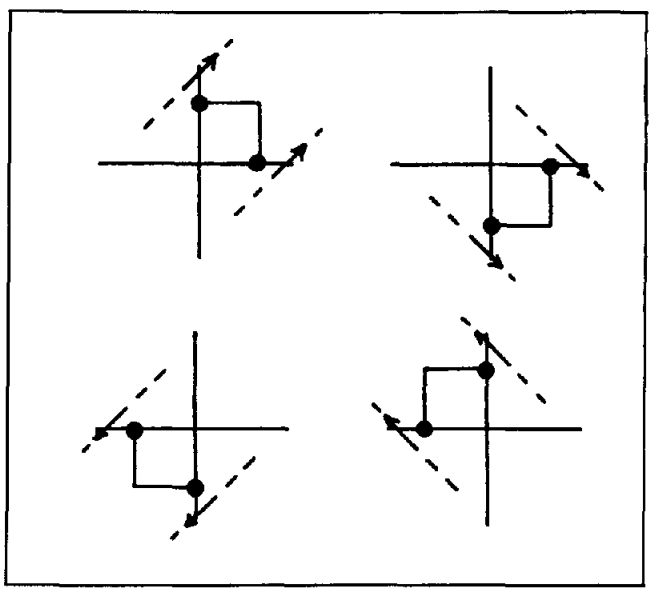

FIGURE 16. Case 6 ($0 < t_{in_2} \leq 1$ and $t_{out_1} < t_{in_2}$): Turning Vertex.

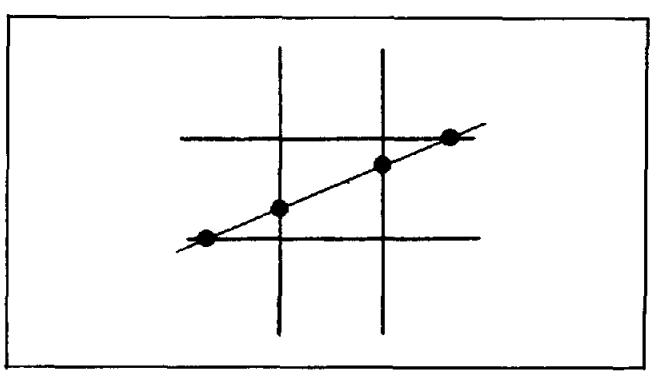

FIGURE 17. The Near-Horizontal Case Passing Through the Window.

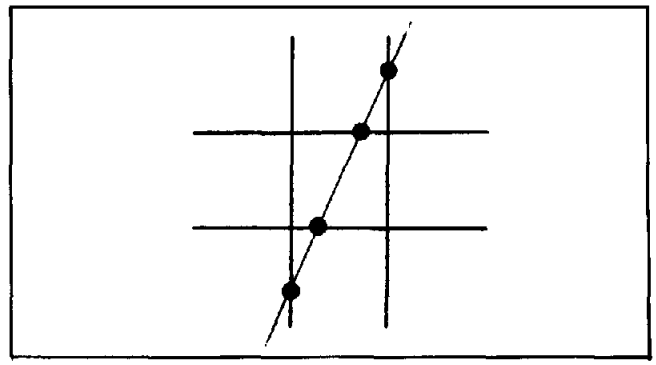

FIGURE 18. The Near-Vertical Case Passing Through the Window.

The analogous analysis holds for the near-vertical case which is shown in Figure 18. This is represented by

$$t_{in_x} \leq t_{in_y} \leq t_{out_y} \leq t_{out_x} \tag{14}$$

Since t_{in_y} and t_{out_y} are finite, the appropriate infinite values are

$$\begin{aligned} t_{in_x} &\rightarrow -\infty \\ t_{out_x} &\rightarrow +\infty \end{aligned} \tag{15}$$

Now, consider the case of l_i bypassing the window. In this case, there are two sets of infinite values that are valid, depending on whether the derivation is based on entering a horizontal or vertical boundary first. Depending on the direction of l_i, Figure 19 can be represented as either

$$t_{in_y} \leq t_{out_y} \leq t_{in_x} \leq t_{out_x} \tag{16}$$

or

$$t_{in_x} \leq t_{out_x} \leq t_{in_y} \leq t_{out_y} \tag{17}$$

For the horizontal case, t_{in_x} and t_{out_x} are finite and t_{in_y} and t_{out_y} are infinite. The infinite values corresponding to inequalities 16 and 17, respectively, are

$$\begin{aligned} t_{in_y} &\rightarrow -\infty \\ t_{out_y} &\rightarrow -\infty \end{aligned} \tag{18}$$

and

$$\begin{aligned} t_{in_y} &\rightarrow +\infty \\ t_{out_y} &\rightarrow +\infty \end{aligned} \tag{19}$$

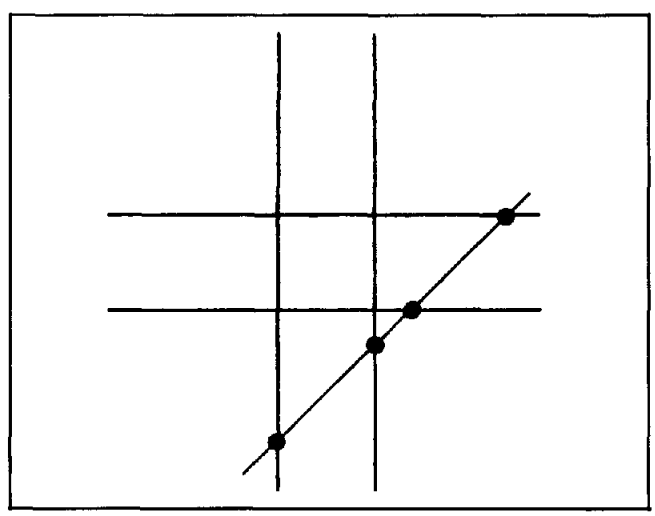

FIGURE 19. l_i Bypassing the Window.

```
x[n + 1] := x[1];
y[n + 1] := y[1];
for i := 1 to n do
begin (* edge V[i]V[i + 1]*)
  deltax := x[i + 1] − x[i];
  deltay := y[i + 1] − y[i];
  if deltax > 0
  then begin (*l[i] points to the right *)
    xin := xleft;
    xout := xright
  end
  else begin (*l[i] points to the left *)
    xin := xright;
    xout := xleft
  end;
  if deltay > 0
  then begin (*l[i] points up *)
    yin := ybottom;
    yout := ytop
  end
  else begin (*l[i] points down *)
    yin := ytop;
    yout := ybottom
  end;
  if deltax <> 0 then tinx := (xin − x[i])/deltax
          else (* vertical *) tinx := −infinity;
  if deltay <> 0 then tiny := (yin − y[i])/deltay
          else (* horizontal *) tiny := −infinity;
  if tinx < tiny
  then begin (* first entry at x then y *)
    tin1 := tinx;
    tin2 := tiny
  end
  else begin (* first entry at y then x *)
    tin1 := tiny;
    tin2 := tinx
  end;
  if 1 > = tin1
  then begin (* case 2 or 3 or 4 or 6 *)
    if 0 < tin1 then(* case 5--turning vertex *) appendvertex
    (xin, yin);
    if 1 > = tin2
    then begin (* case 3 or 4 or 6 *)
      if deltax <> 0 then toutx := (xout − x[i])/deltax
              else (* vertical *)
                if (xleft < = x[i]) and (x[i] < = xright) then
                                    (* inside *)
                                    toutx := infinity
                                    else (* outside *)
                                    toutx := −infin-
                                    ity;
      if deltay <> 0 then touty := (yout − y[i])/deltay
              else (* horizontal *)
      if (ybottom < = y[i]) and (y[i] < = ytop)
                    then (* inside *)
                    touty := infinity
                    else (* outside *)
                    touty := −infinity;
      if toutx < touty then (* first exit at x *) tout1 := toutx
        else (* first exit at y *) tout1 := touty;
      if (0 < tin2) or (0 < tout1)
      then begin (* case 4 or 6 *)
        if tin2 < = tout1
        then begin (* case 4 -- visible segment *)
          if 0 < tin2
          then (* V[i] outside window *)
              if tinx > tiny then (* vertical boundary *)
                      appendvertex
                      (xin, y[i] + tinx * deltay)
              else (* horizontal boundary *)
              appendvertex (x[i] + tiny * deltax, yin);
          if 1 > tout1
          then (* V[i + 1] outside window *)
              if toutx < touty then (* vertical boundary *)
                      appendvertex
                      (xout, y[i] + toutx * deltay)
              else (* horizontal boundary *)
              appendvertex (x[i] + touty * deltax, yout)
          else (* V[i + 1] inside window *) appendvertex
          (x[i + 1], y[i + 1])
        end (* case 4 *)
        else (* case 6 -- turning vertex *) if tinx > tiny then
                                (* second entry at x *)
                                appendvertex (xin, yout)
          else (* second entry at y *) appendvertex
          (xout, yin)
      end (* case 4 or 6 *)
    end (* case 3 or 4 or 6 *)
  end (* case 2 or 3 or 4 or 6 *)
end (* edge V[i]V[i + 1] *).
```

FIGURE 20. The Algorithm Coded in Pascal.

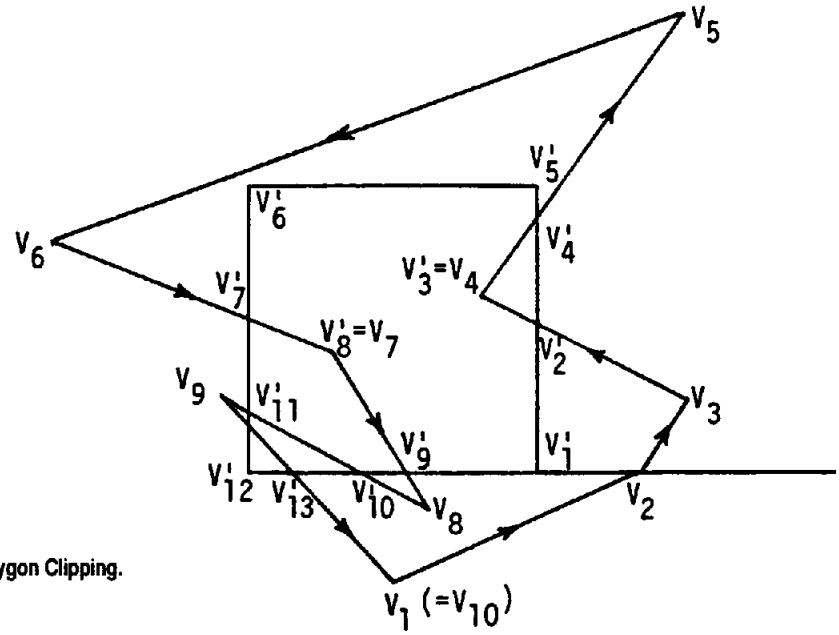

FIGURE 21. Example of Polygon Clipping.

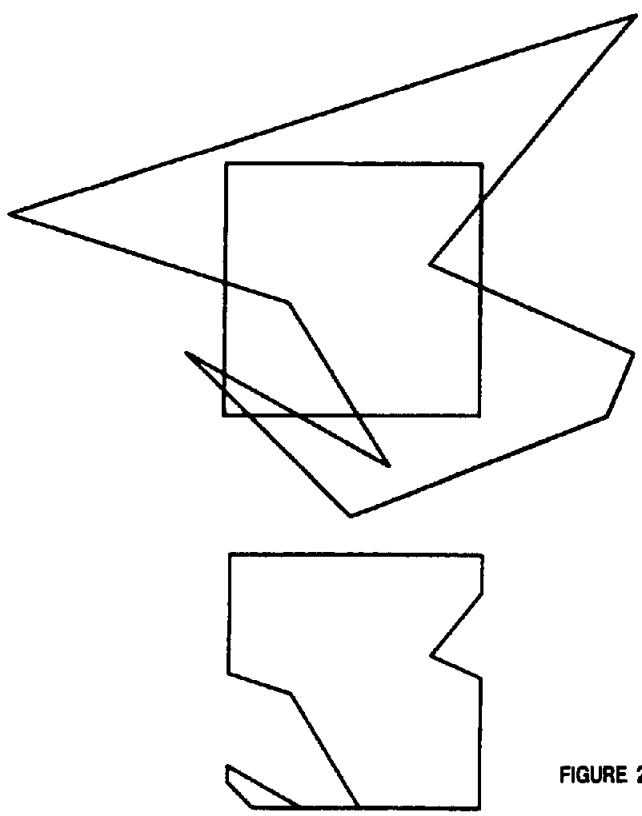

FIGURE 22. Display Showing the Example Input Polygon

For the vertical case, t_{in_y} and t_{out_y} are finite and t_{in_x} and t_{out_x} are infinite. The corresponding infinite values are, respectively,

$$t_{in_x} \rightarrow +\infty$$

$$t_{out_x} \rightarrow +\infty \tag{20}$$

and

$$t_{in_x} \rightarrow -\infty$$

$$t_{out_x} \rightarrow -\infty \tag{21}$$

The choice of which set of infinite values to use is completely arbitrary. The actual decision is based on a slight efficiency that could be exploited if the $-\infty$ values were used. Due to the manner in which the actual algorithm is structured, t_{in_x} and t_{in_y} are always computed whereas t_{out_x} and t_{out_y} are not calculated in Cases 1 or 2. When I_i passes through the window, $t_{in_y} \rightarrow -\infty$ if I_i is horizontal and $t_{in_x} \rightarrow -\infty$ if I_i is vertical. Thus, the simplest approach is to use the $-\infty$ values for the case where I_i bypasses the window. In this manner, the check for whether I_i passes inside or outside the window need only be done for Cases 3 through 6.

6. ALGORITHM

Figure 20 shows the algorithm coded in Pascal. The ordering of the cases has been reorganized to avoid redundant comparisons. Note that Case 5 is dealt with prior to Case 4 or Case 6, as was explained in Sec. 4. If none of Cases 4, 5, and 6 is applicable, no output is generated.

The computational complexity of the algorithm is linear in the number of edges. The exact requirements are dependent upon the particular data. In general, the worst case for a single edge occurs when both of its endpoints lie outside the window and it has a visible segment. In this case, two intersection points must be computed and included in the output polygon. This requires a total of 18 comparisons, 8 additions/subtractions, 2 multiplications, and 4 divisions.

7. EXAMPLE

As an example, consider the closed polygon $V_1V_2V_3V_4V_5V_6V_7V_8V_9V_{10}(= V_1)$ with V_2 lying on the boundary line, as shown in Figure 21. Table III shows for each edge of the input polygon, the type of contribution to the output polygon as well as the actual output. Since the input polygon is closed, the initial and final vertices of the output polygon will be on the same boundary of the window, and thus must be connected as a final step in the algorithm.

This example was computed using both this algorithm as well as the traditional Sutherland–Hodgman [3] reentrant polygon clipping algorithm. The former algorithm required only half the execution time of the latter. (Specifically, they required 5 and 10 seconds cpu time, respectively, to perform the clipping 100 times on a DEC VAX-11/780 in Pascal under Berkeley UNIX.) The results of both algorithms are identical: both produce a single polygon containing a degenerate edge (Figure 22).

TABLE III. The Type of Contribution and the Actual Output for Each Edge of the Example Input Polygon

Edge	Contribution	Actual Output
V_1V_2	Turning vertex	V'_1
V_2V_3	$n+1$	$n+1$
V_3V_4	Visible segment	$V'_2V'_3$
V_4V_5	Visible segment	V'_4
V_5V_6	Two turning vertices	$V'_5V'_6$
V_6V_7	Visible segment	$V'_7V'_8$
V_7V_8	Visible segment	V'_9
V_8V_9	Visible segment	$V'_{10}V'_{11}$
V_9V_{10}	Visible segment	$V'_{12}V'_{13}$

8. CONCLUSIONS

A very direct algorithm has been developed by carefully analyzing the process of polygon clipping. In this algorithm, each edge of the polygon is treated independently (and hence could be processed in parallel). Each is considered to be a vector and is extended to be a line of infinite extent. Similarly, each of the boundary line segments is extended to be a line of infinite extent. The intersection points of an extended edge with the boundary lines are identified as *entry* or *exit* points. An important observation is that this identification can be used to characterize the visibility status of the edge. Regardless of whether the edge has a visible segment, it might induce the inclusion in the output polygon of a vertex at a corner of the clip window. This is called a *turning vertex* because the output polygon "turns" at this point.

These concepts were formalized with the introduction of some notation. The edge was expressed as parametric equations. Using this notation, the possible contributions (including nil) that can be provided to the output polygon by a particular edge of the input polygon were enumerated.

After extending this development to handle the case of horizontal or vertical edges, the analysis was embedded in an algorithm coded in Pascal. An arbitrary example input polygon is given and the type of contribution and the actual output for each edge of the polygon is shown. In the example, this algorithm required only half the execution time than that of the traditional Sutherland–Hodgman reentrant polygon clipping algorithm.

Acknowledgments. The authors would like to thank Shinichiro Haruyama for his programming effort, Michael H. Arnold and Anthony D. DeRose for their assistance, Douglas B. Terry for his interest, and the other members of the computer graphics class at Berkeley for their enthusiasm during the development of this analysis and algorithm. Appreciation is also expressed to Ivan E. Sutherland and to Kevin J. Weiler for taking the time to read a draft of this paper.

REFERENCES

1. Liang You-Dong and Barsky, Brian A. A new concept and method for line clipping. *ACM Transactions on Graphics*, to appear.
2. Liang You-Dong and Barsky, Brian A. Three new algorithms for two-dimensional line clipping. (In preparation.)
3. Sutherland Ivan E. and Hodgman, Gary W. Reentrant polygon clipping. *Comm. ACM. 17*, 1 (Jan. 1974) 32–42.
4. Tilove, Robert B. Set membership classification: A unified approach to geometric intersection problems. *IEEE Trans. Computers C-29*, 10 (Oct. 1980) 874–883.
5. Weiler, Kevin J. *Hidden Surface Removal Using Polygon Area Sorting*. Master's Thesis, Cornell University, Ithaca, N.Y. (Jan. 1978).
6. Weiler, Kevin J. Polygon comparison using a graph representation. *Proc. SIGGRAPH '80*, ACM, (July, 1980), pp. 10–18. Also Research Report No. 6, Institute of Building Sciences, Carnegie-Mellon University, Pittsburgh, Pennsylvania, May 1980.

CR Categories and Subject Descriptors: I.3.3 [**Computer Graphics**]: Picture/Image Generation—*display algorithms*; I.3.4 [**Computer Graphics**]: Graphics Utilities—*software support*; I.3.5 [**Computer Graphics**]: Computational Geometry and Object Modeling—*geometric algorithms, languages, and systems*

General Term: Algorithms

Additional Key Words and Phrases: clipping, polygon clipping

Received 3/82; revised 11/82; accepted 11/82